

6.5830/1 Lecture 15: Distributed Databases

4/7/2026

Tianyu Li

litianyu@mit.edu

Many thanks to Andy Pavlo
and CMU DB for sharing
their materials

Administrivia

- Pset 3 will be out later today
- Project Meetings + midterm reports (4/14)
 - If you were emailed about changing your project, take free late days on this one
- Lab 3 due next week (4/16)
 - Writeup 4/21

Last Class: ARIES

- Buffer Pool Policy: **STEAL** + **NO-FORCE**
- Write ahead logging
 - Log before page flush
 - Record LSN on page
- Fuzzy Checkpointing
 - Record ATT & DPT
 - Flush all dirty pages periodically
- Recovery: Analysis + Redo + Undo
 - Undo must generate CLR

Last Class

- What if:
 - Disk on machine fails
 - Computer won't restart
 - Missile hits your data center
 - ...
- Solution: backup machines
 - Database becomes a distributed system!

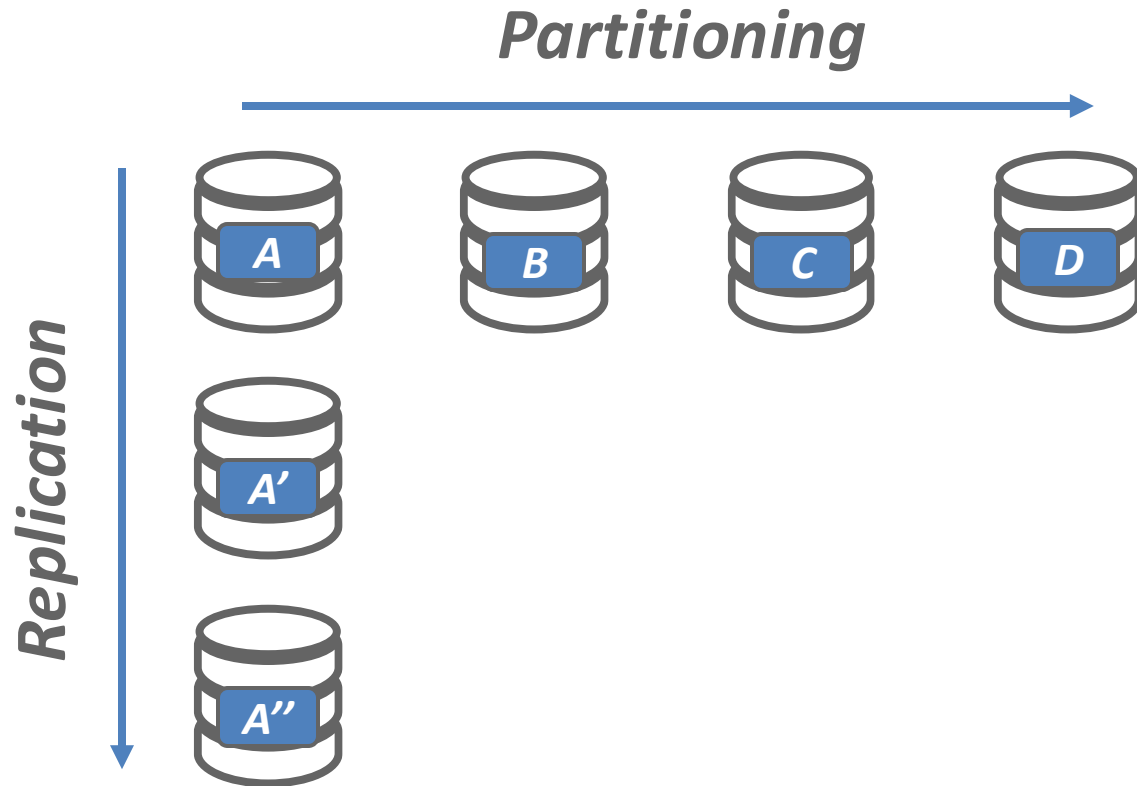


Distributed DBMS

Two reasons to distribute (not mutually exclusive):

- For improved performance & scalability (parallel DB)
 - Nodes are physically close to each other.
 - Nodes connected with high-speed LAN.
 - Communication cost is assumed to be small.
- For improved availability & durability (distributed DB)
 - Nodes can be far from each other.
 - Nodes connected using public network.
 - Communication cost and problems cannot be ignored.

Distribution Primitives

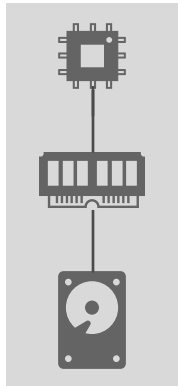


Distributed DBMS Architecture

- A distributed DBMS's system architecture specifies what shared resources are directly accessible to CPUs.
- This affects how CPUs coordinate with each other and where they retrieve/store objects in the database.

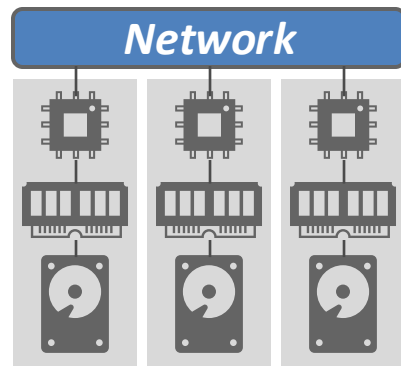
Distributed DBMS Architecture

Shared-Everything



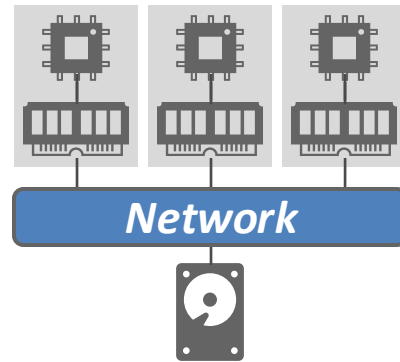
↑ *i.e., single machine*

Shared-Nothing



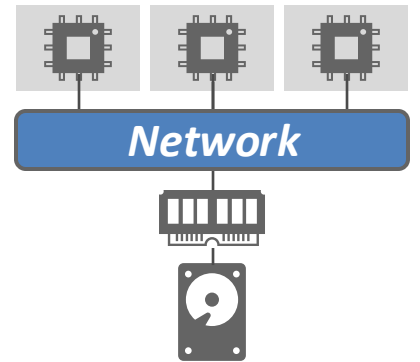
↑ *Common (Decreasing)*

Shared-Disk



↑ *Common (Increasing)*

Shared-Memory

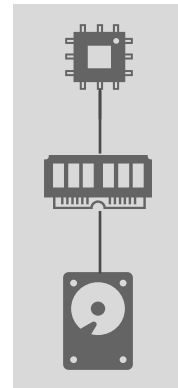


↑ *CXL?*

Shared Everything

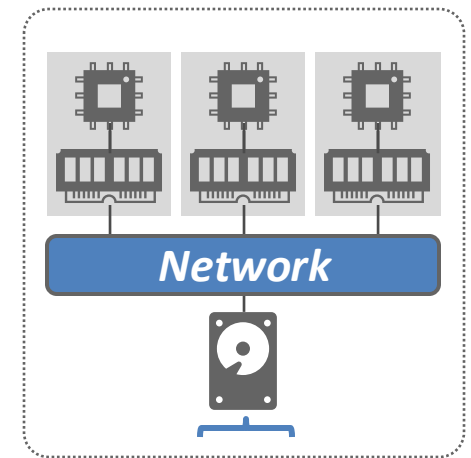
- Conventional multicore computer
 - Perhaps virtualized
- Each core can access any record
 - Difficult to scale beyond a few cores
- Not fault tolerant

*Shared-
Everything*



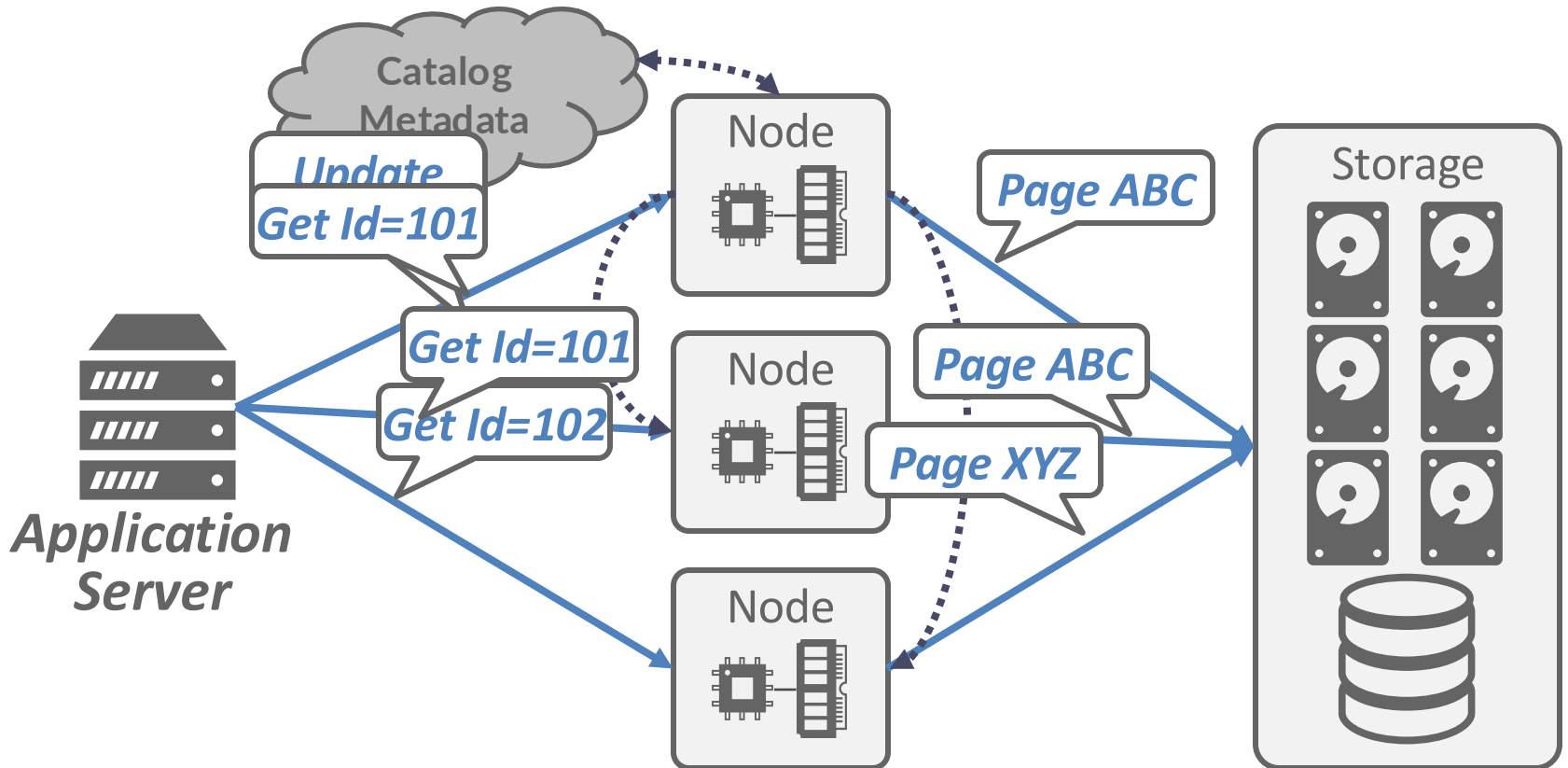
Shared Disk

- Nodes access a single logical disk via an interconnect, but each have their own private memories.
 - Scale execution layer independently from the storage layer.
 - Nodes can still use direct attached storage as a slower/larger cache.
 - This architecture was not popular due to needing specialized hardware and complex coherence protocol
 - This architecture is seeing a renaissance in the cloud thanks to modern **data lakes** and **serverless** systems.



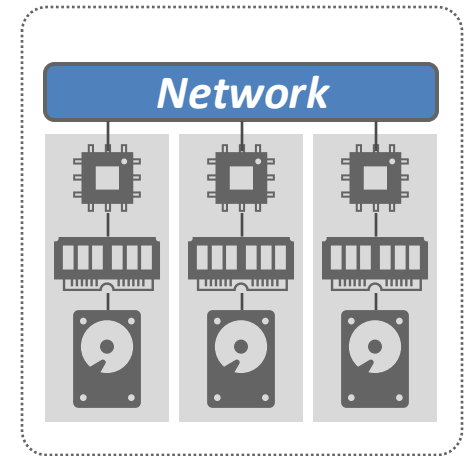
*Distributed File Systems
Object Stores*

Example



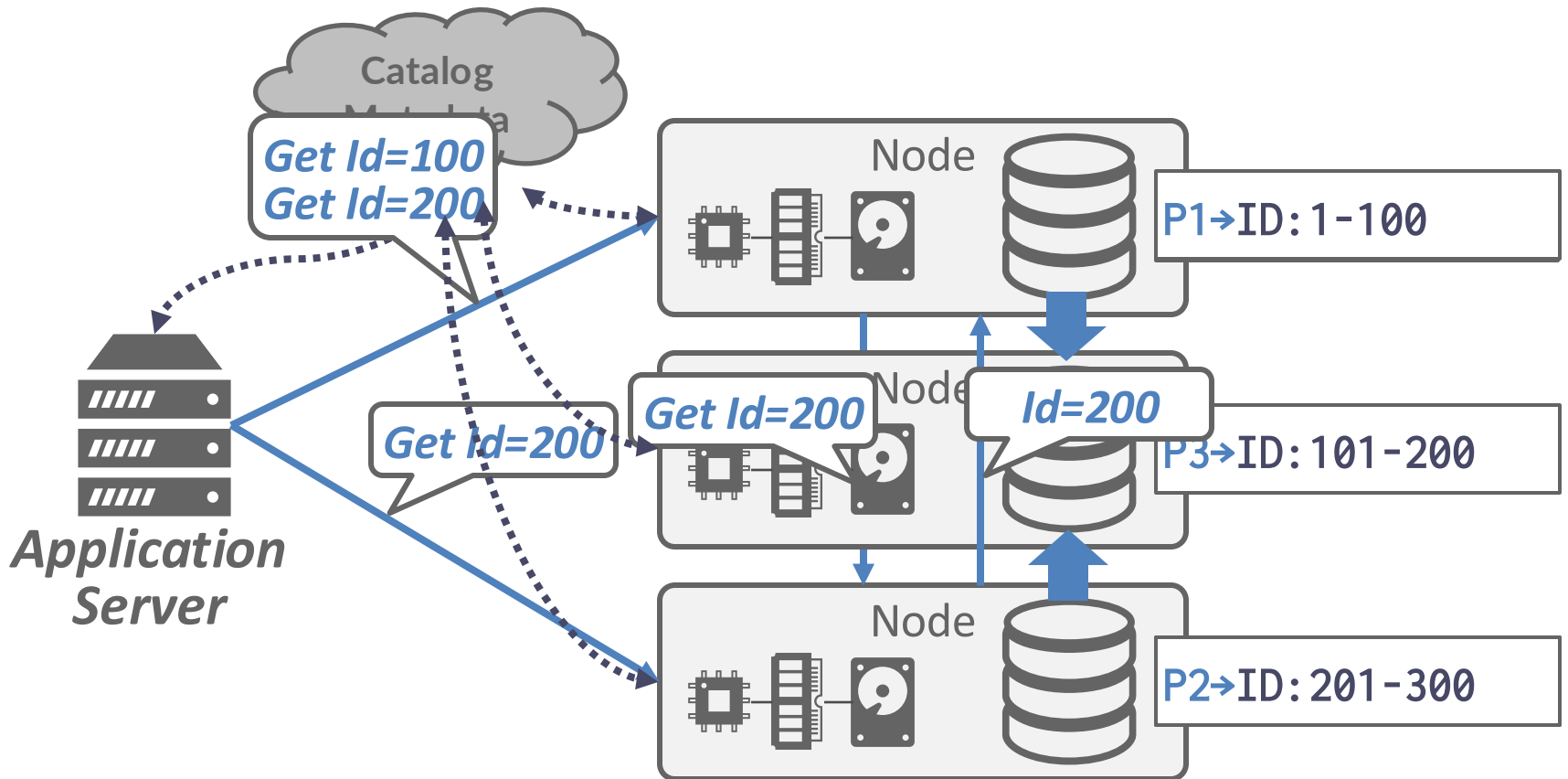
Shared Nothing*

- Each DBMS node has its own CPU, memory, and local disk.
- Nodes only communicate with each other via network.
 - Better performance & efficiency.
 - Harder to scale capacity.
 - Harder to ensure consistency.
- Gold-standard before the cloud era
 - Still very commonplace



* Rumor has it that Mike Stonebraker is in a bluegrass band named Shared Nothings with other database faculties

Example



Distributed DBMS

Two reasons to distribute (not mutually exclusive):

- For improved performance & scalability (parallel DB)
 - Nodes are physically close to each other.
 - Nodes connected with high-speed LAN.
 - Communication cost is assumed to be small.
- For improved availability & durability (distributed DB)
 - Nodes can be far from each other.
 - Nodes connected using public network.
 - Communication cost and problems cannot be ignored.

Parallel DB Goal

- SQL, but faster by running on multiple processors
- What do we mean by faster?

$$\text{speed up} = \frac{\text{old time}}{\text{new time}}$$

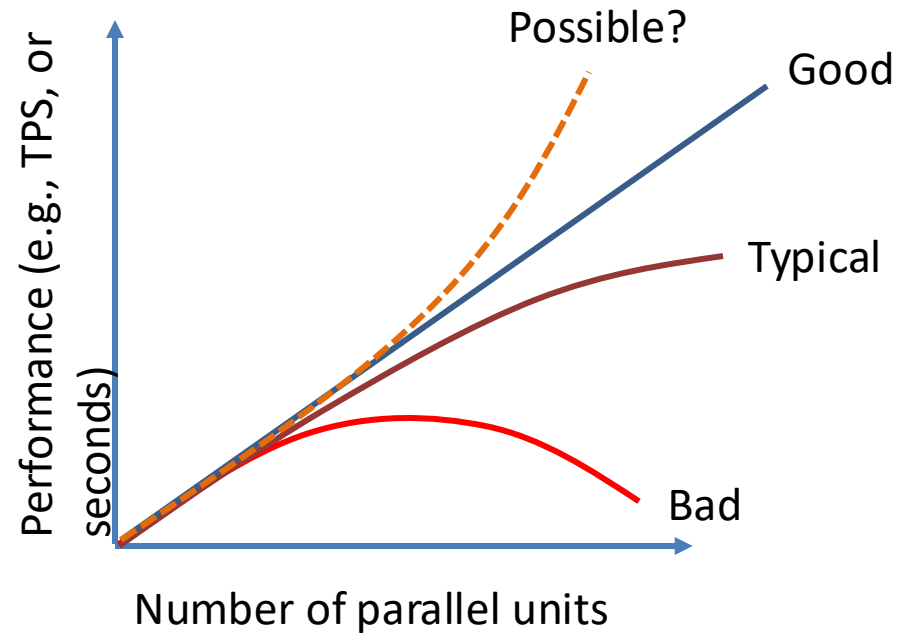
on same problem, with N times more hardware

$$\text{scale up} = \frac{1x \text{ larger problem on } 1x \text{ hardware}}{Nx \text{ larger problem on } Nx \text{ hardware}}$$

- Not necessarily the same: smaller problem may be harder to parallelize

Speedup Goal

- Linear?



Barriers to Linear Scaling

What are some barriers in analytics and transactional workloads?

- Startup times
 - e.g., may take time to launch each parallel executor
- Interference
 - processors depend on some shared resource
 - E.g., input or output queue, or other data item
- Amdahl's Law
 - Some portion of the program not parallelizable

Parallel DB

- Idea: run multiple tasks simultaneously to take advantage of hardware parallelism
- Thanks to the relational model, this is largely transparent to the application and is purely a decision within the DBMS

Inter-Query Parallelism

- Multiple database queries run in parallel
 - We have been implicitly assuming this model
- If queries are read-only, then this requires almost no explicit coordination between the queries.
 - Buffer pool can handle most of the sharing if necessary.
- Otherwise, must rely on concurrency control
 - Which may introduce contention and bottlenecks
 - Challenging in a distributed system

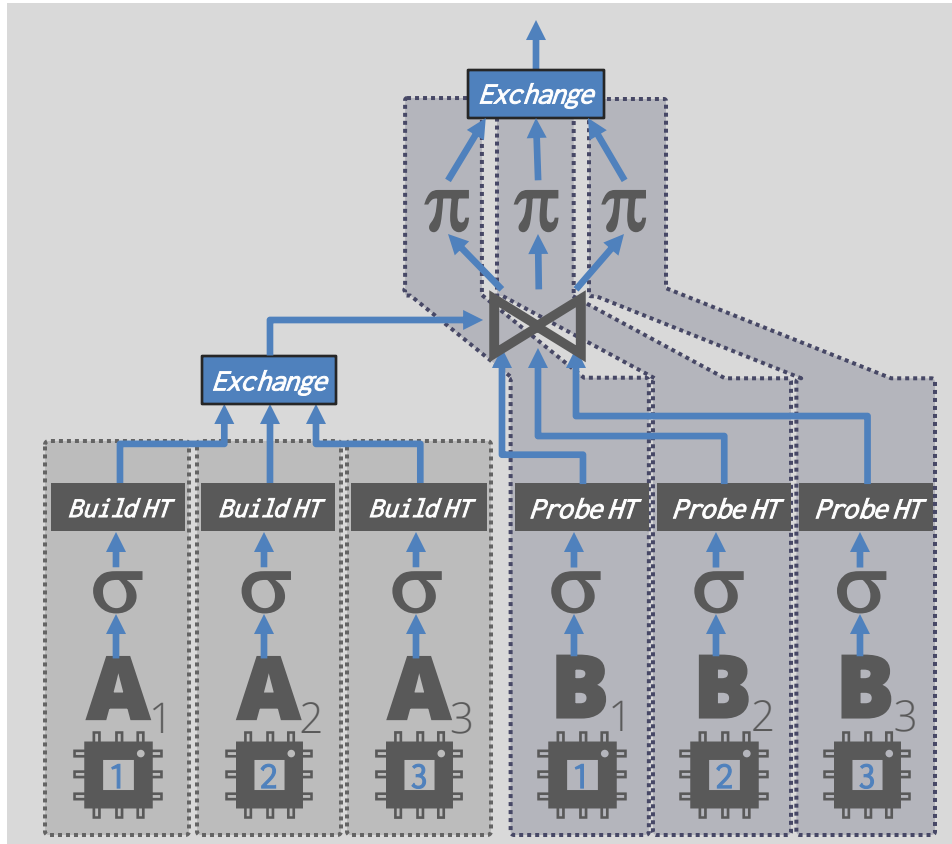
Intra-Query Parallelism

- Improve the performance of a single query by executing its operators in parallel.
 - Think of the organization of operators in terms of a producer/consumer paradigm.
- **Approach #1: Intra-Operator**
- **Approach #2: Inter-Operator**
- These techniques are not mutually exclusive.

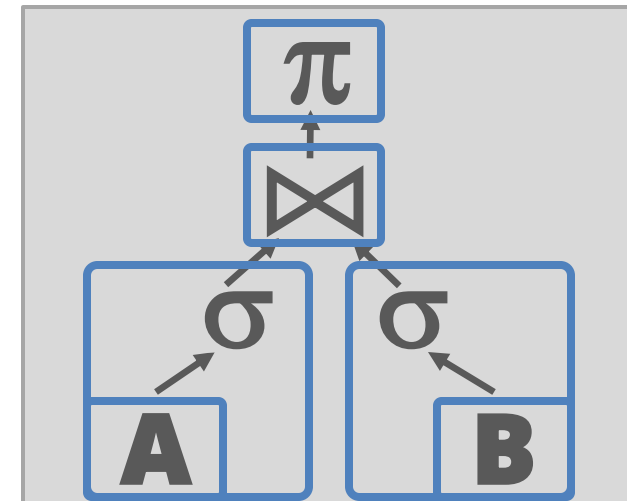
Intra-Operator Parallelism

- **Approach #1: Intra-Operator**
 - Operators are decomposed into independent instances that perform the same function on different subsets of data.
- The DBMS inserts an **exchange** operator into the query plan to coalesce/split results from multiple children/parent operators.
 - Can combine intermediate results in arbitrary order or merge them according to a sorting key.

Intra-Operator Parallelism

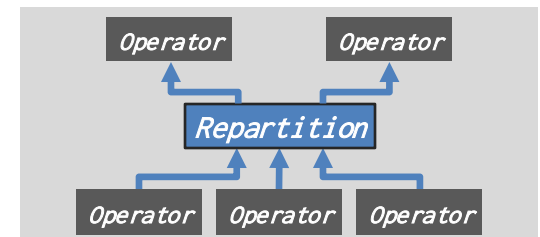
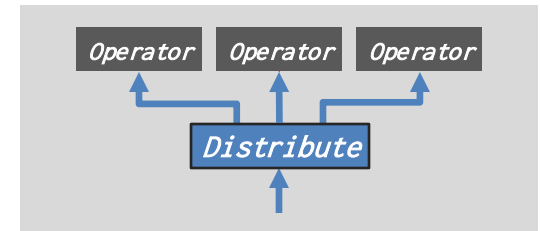
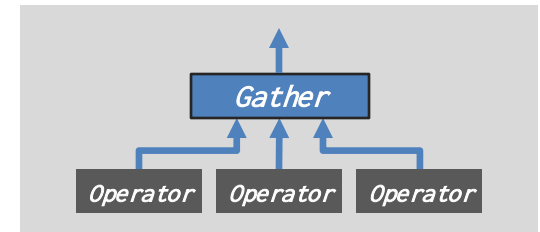


```
SELECT A.id, B.value
FROM A JOIN B
      ON A.id = B.id
WHERE A.value < 99
      AND B.value > 100
```



Exchange Operator

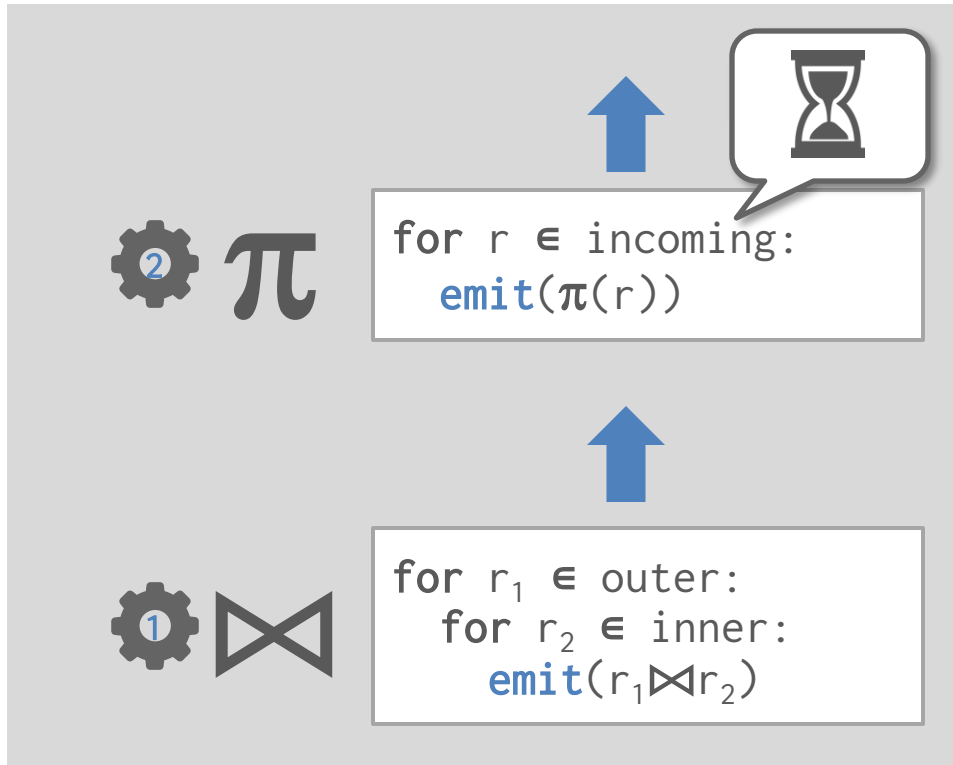
- **Exchange Type #1 – Gather**
 - Combine the results from multiple workers into a single output stream.
- **Exchange Type #2 – Distribute**
 - Split a single input stream into multiple output streams.
- **Exchange Type #3 – Repartition**
 - Shuffle multiple input streams across multiple output streams.



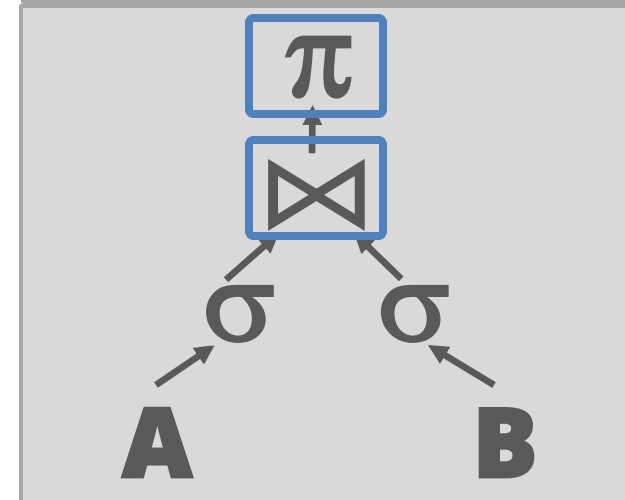
Inter-Operator Parallelism

- **Approach #2: Inter-Operator**
 - Operations are overlapped to pipeline data from one stage to the next without materialization.
 - Workers execute multiple operators from different segments of a query plan at the same time.
 - Still need exchange operators to combine intermediate results from segments.
- Also called **pipelined parallelism**.

Inter-Operator Parallelism



```
SELECT SLOW_UDF(B.value)
FROM A JOIN B
      ON A.id = B.id
WHERE A.value < 99
      AND B.value > 100
```



Problem

- Only works when each pipeline stage is about the same speed
- Limited parallelism as most pipelines are short
- Data movement can be more expensive than parallelism speed-up
- As a result, intra-operator parallelism is the primary way database systems scale

Observation

- Using multiple workers for parallel query execution will not improve the DBMS's performance if storage is the bottleneck.
- It can sometimes make the DBMS's performance worse if a worker is accessing different segments of the disk at the same time.

Partitioning Schemes

- Partition storage in addition to compute
 - Called "sharding" in NoSQL systems.
- The DBMS executes query fragments on each partition and then combines the results to produce a single answer.
 - Ideally, partitioning is transparent to the application

Strawman

- Assign an entire table to a single node.
- Assumes that each node has enough storage space for an entire table.
- Ideal if queries never join data across tables stored on different nodes and access patterns are uniform.

Strawman

Table1

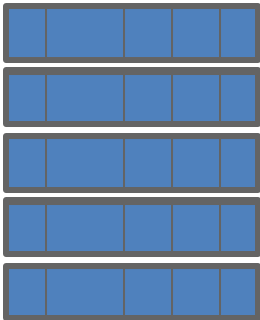
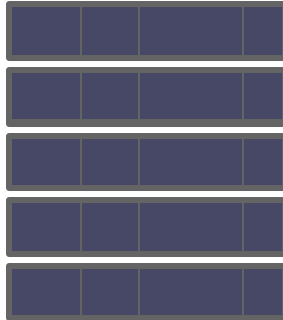
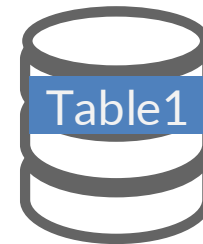


Table2



Partitions



Ideal Query:

```
SELECT * FROM table1
```

Horizontal Partitioning

- Split a table's tuples into disjoint subsets based on some partitioning key and scheme.
 - Choose column(s) that divides the database equally in terms of size, load, or usage.
- Partitioning Schemes:
 - Hashing
 - Ranges
 - Predicates
 - Round Robin

Horizontal Partitioning

Partitioning Key

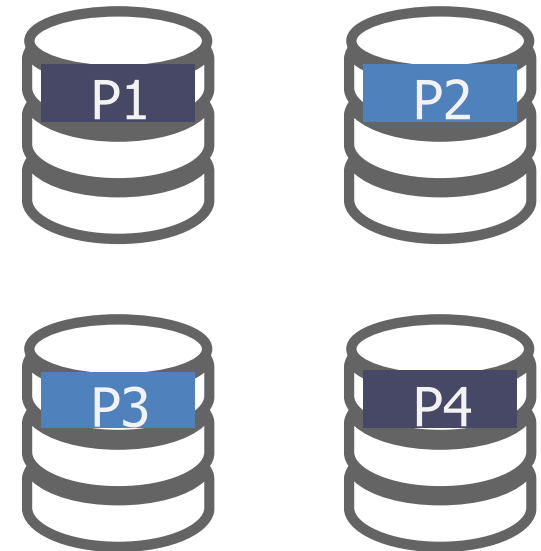
Table

101	a	XX	2025-11-29	$\text{hash}(a)\%4 = P2$
102	b	XY	2025-11-28	$\text{hash}(b)\%4 = P4$
103	c	YZ	2025-11-29	$\text{hash}(c)\%4 = P3$
104	d	YX	2025-11-27	$\text{hash}(d)\%4 = P2$
105	e	YY	2025-11-29	$\text{hash}(e)\%4 = P1$

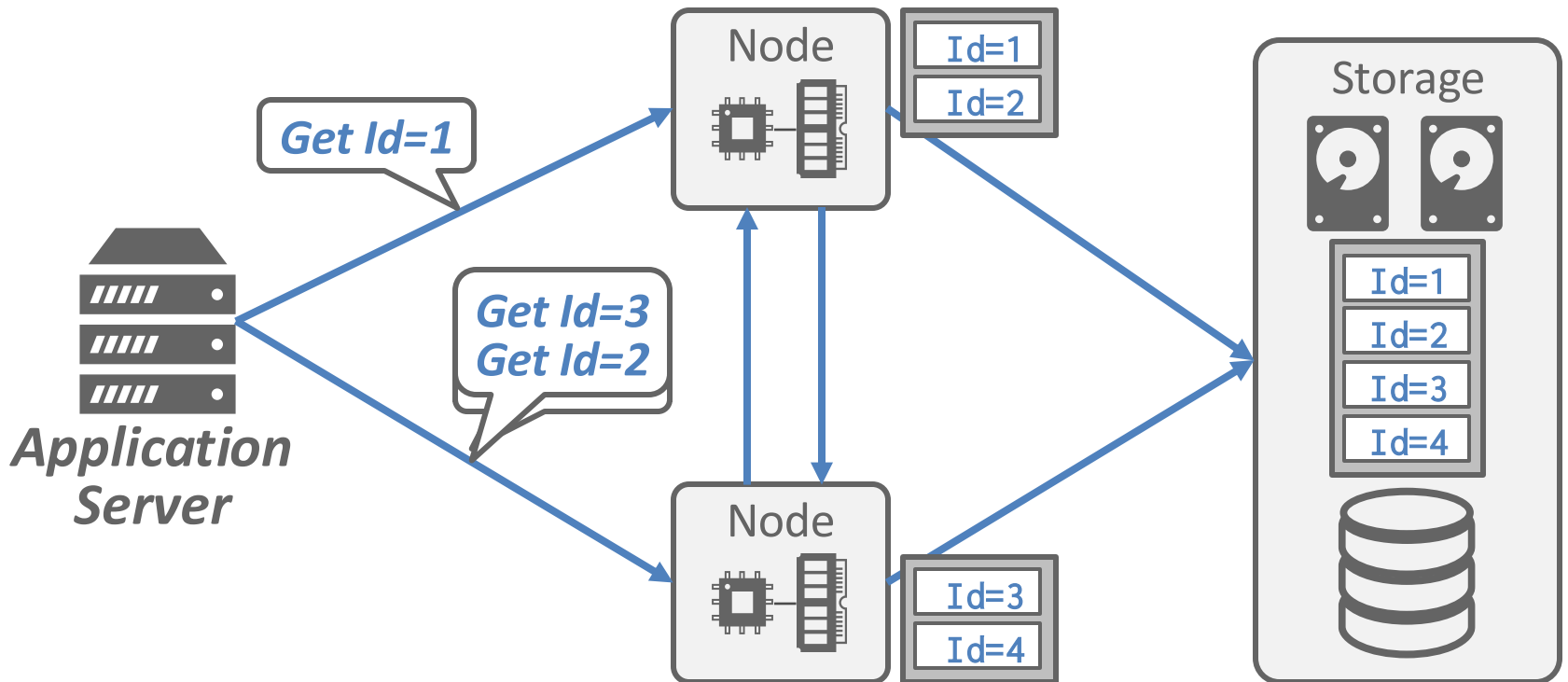
Ideal Query:

```
SELECT * FROM table
WHERE partitionKey = ?
```

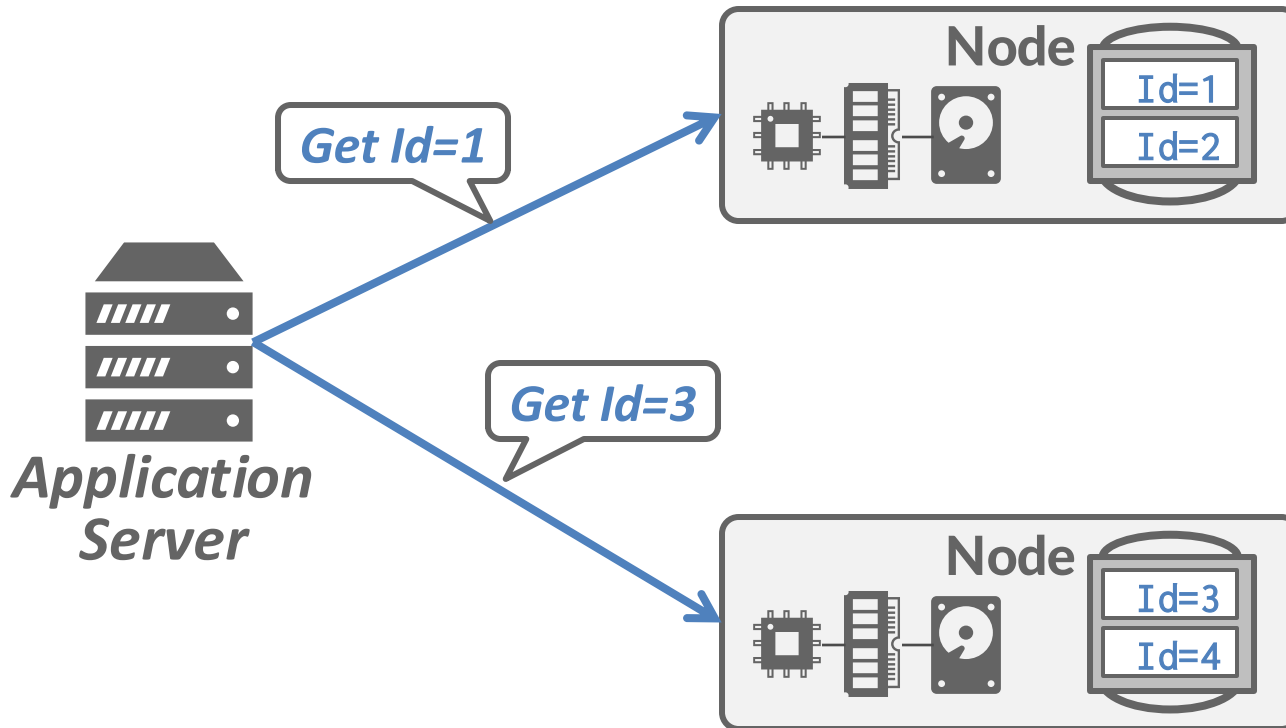
Partitions



Shared-Disk Partitioning



Shared-Disk Partitioning



Problems?

Partitioning Key

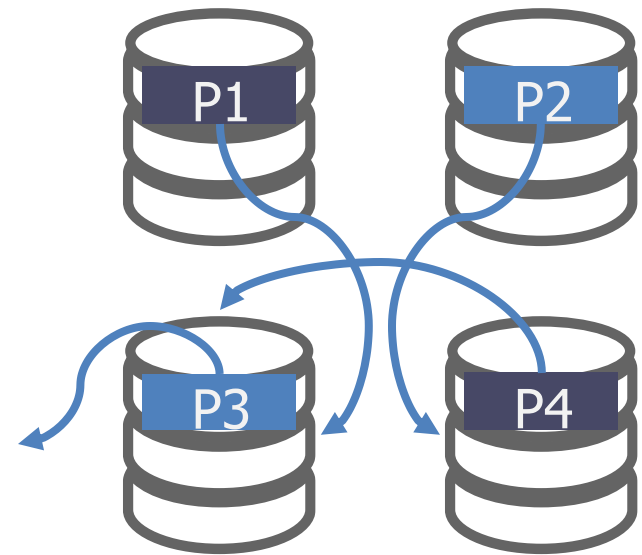
Table

101	a	XXX	2025-11-29	$\text{hash}(a)\%3 = P2$
102	b	XXY	2025-11-28	$\text{hash}(b)\%3 = P3$
103	c	XYZ	2025-11-29	$\text{hash}(c)\%3 = P3$
104	d	XYX	2025-11-27	$\text{hash}(d)\%3 = P2$
105	e	XYY	2025-11-29	$\text{hash}(e)\%3 = P3$

Ideal Query:

```
SELECT * FROM table
WHERE partitionKey = ?
```

Partitions



Parallel Operations in a Partitioned DB

- **PROJECT, FILTER, etc.**
 - Easy to “push down” to each worker
- **JOIN**
 - Depending on data partitioning, may be able to process partitions individually and then merge, or may need to repartition
- **AGGREGATE**
 - Partially aggregate data at each node, merge final result

Distributed Joins

- To join tables **R** and **S**, the DBMS needs to get the proper tuples on the same node.
- Once the data is at the node, the DBMS then executes the same join algorithms that we discussed earlier in the semester.
 - Need to produce the correct answer as if all the data is located in a single node system.

Scenario #1

- The entire copy of one data set is replicated at every node.
 - Think of it as a small dimension table.
- Each node joins its local data in parallel and then sends their results to a coordinating node.

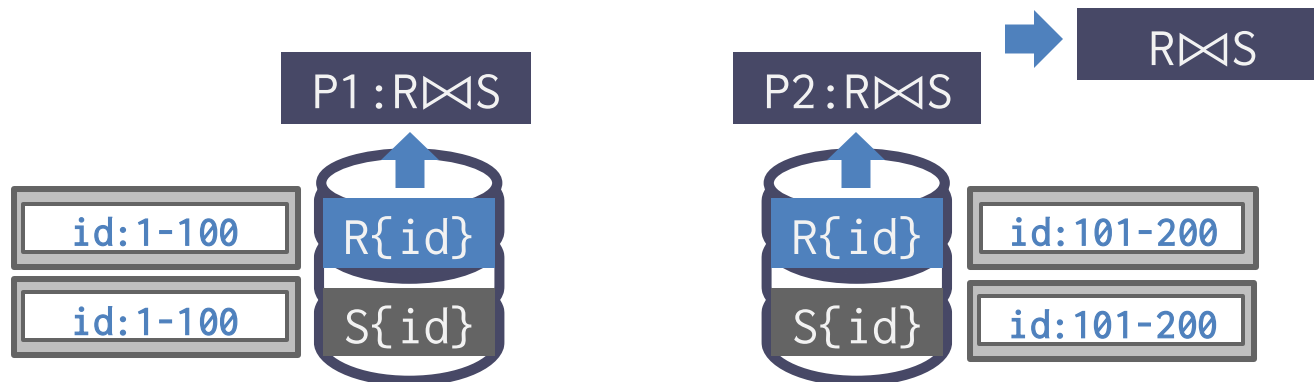
```
SELECT * FROM R JOIN S
ON R.id = S.id
```



Scenario #2

- Both data sets are partitioned on the join attribute. Each node performs the join on local data and then sends to a coordinator node for coalescing.

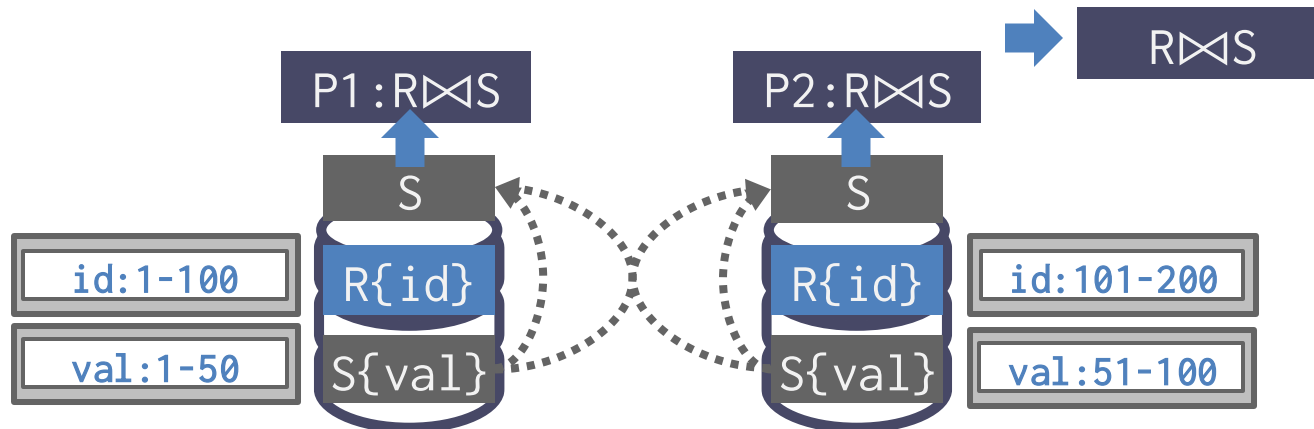
```
SELECT * FROM R JOIN S  
ON R.id = S.id
```



Scenario #3: Broadcast Join

- Both data sets are partitioned on different keys. If one of the data sets is small, then the DBMS "broadcasts" that data to all nodes.

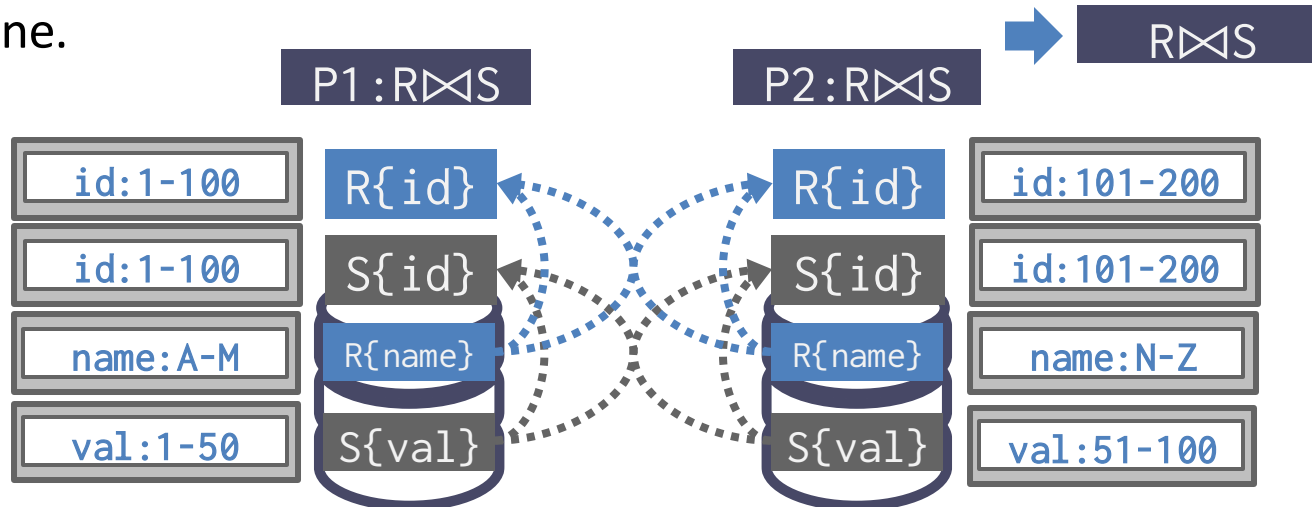
```
SELECT * FROM R JOIN S  
ON R.id = S.id
```



Scenario #4: Shuffle Join

- Both data sets are not partitioned on join key. The DBMS re-partitions the data on-the-fly across nodes.
 - The repartitioned data copy is generally deleted when the query is done.

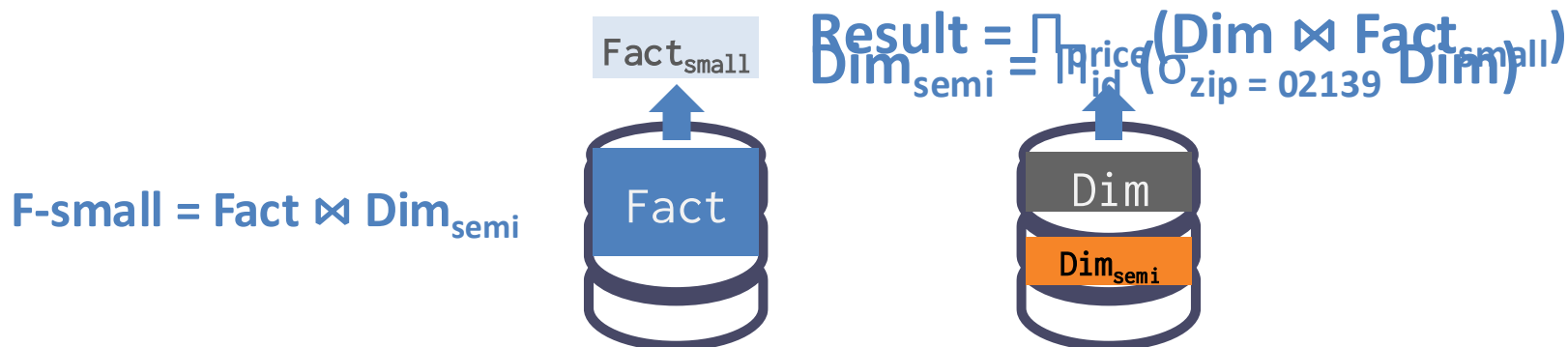
```
SELECT * FROM R JOIN S
ON R.id = S.id
```



Semi-Joins

- Before pulling data from another node, send a **semi-join filter** to reduce data movement.
 - Perform a join on the bare minimum data needed to avoid unnecessary transfers.
 - Could use an approximate filter (Bloom Join).

```
SELECT Fact.price, Dim.*
FROM Fact JOIN Dim
ON Fact.id = Dim.id
WHERE Dim.zip = 02139
```

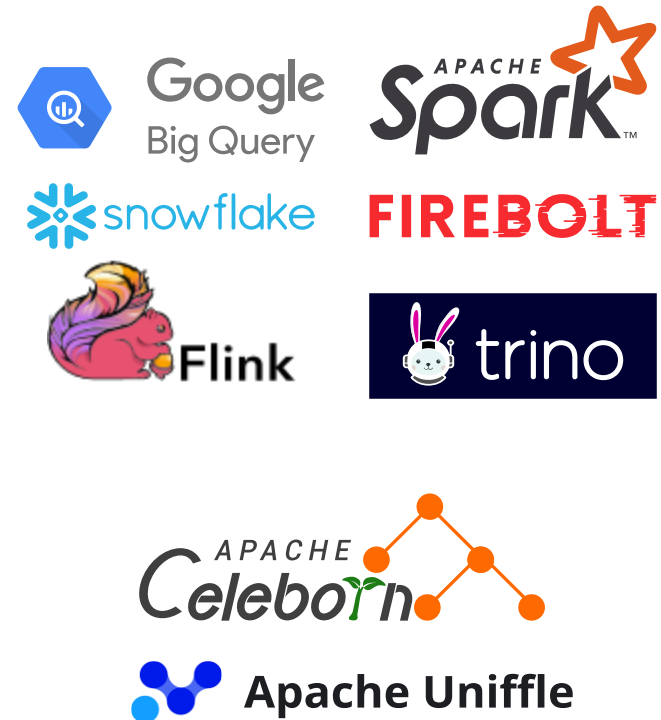


Observation

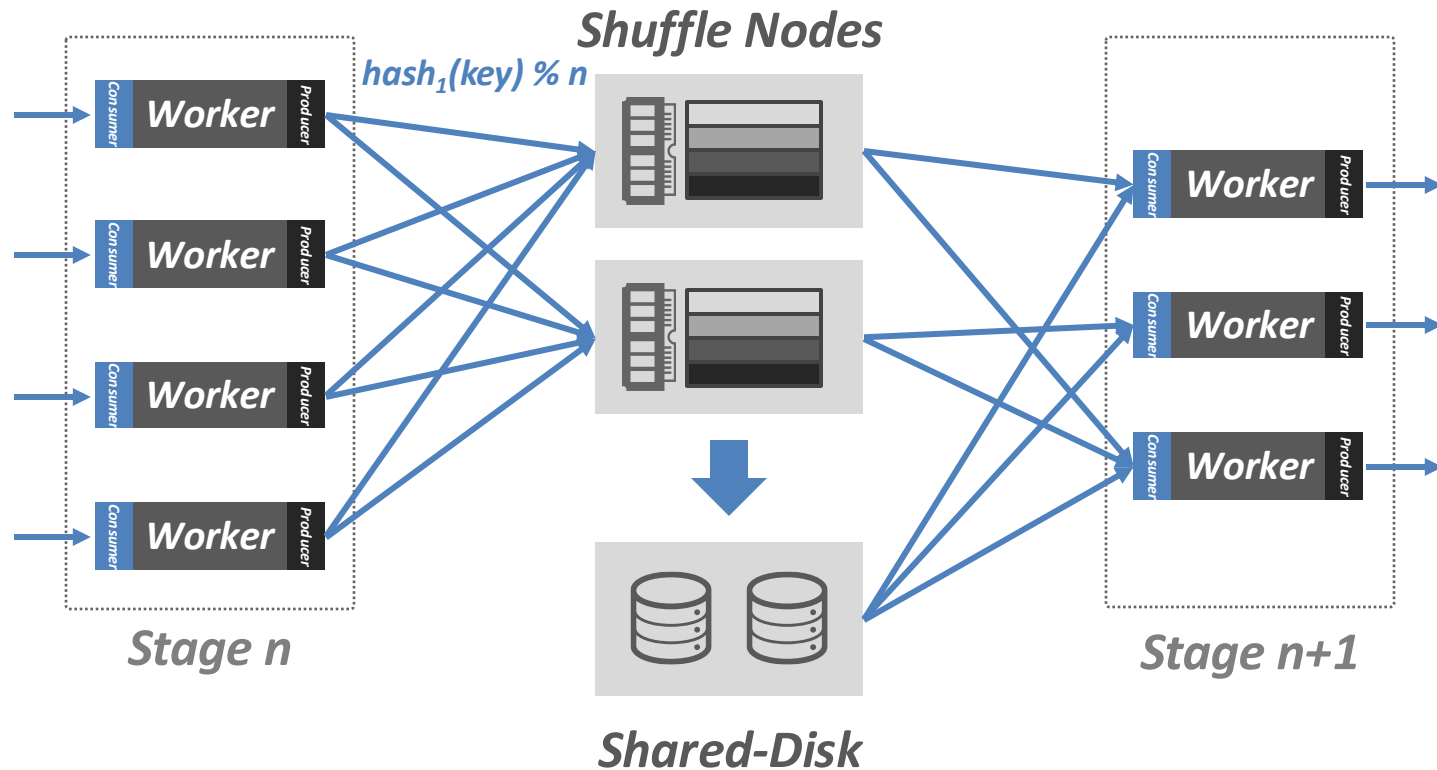
- Direct communication between compute nodes means the DBMS knows which nodes will participate in query execution ahead of time.
- But data skew can cause imbalances...
- A better approach is to dynamically adjust compute resources on the fly as a query executes.

Shuffling

- Redistribute of intermediate data across nodes between query plan pipelines.
 - Can repartition / rebalance data based on observed data characteristics.
- Some DBMSs support standalone fault-tolerant shuffle services.
 - Example: You can replace Spark's built-in in-memory shuffle implementation or replace it with a separate service.



Shuffling



Summary: Parallel DB

- Shared nothing partitioned parallelism is traditionally the dominant approach
 - Shared-disk is increasingly popular in the cloud
- Relational model is key for success
 - Apps don't change when you parallelize system
 - However, for performance tuning, workload-insights are often necessary

Distributed Databases

Two reasons to distribute (not mutually exclusive):

- For improved performance & scalability (parallel DB)
 - Nodes are physically close to each other.
 - Nodes connected with high-speed LAN.
 - Communication cost is assumed to be small.
- For improved availability & durability (distributed DB)
 - Nodes can be far from each other.
 - Nodes connected using public network.
 - Communication cost and problems cannot be ignored.

Replication

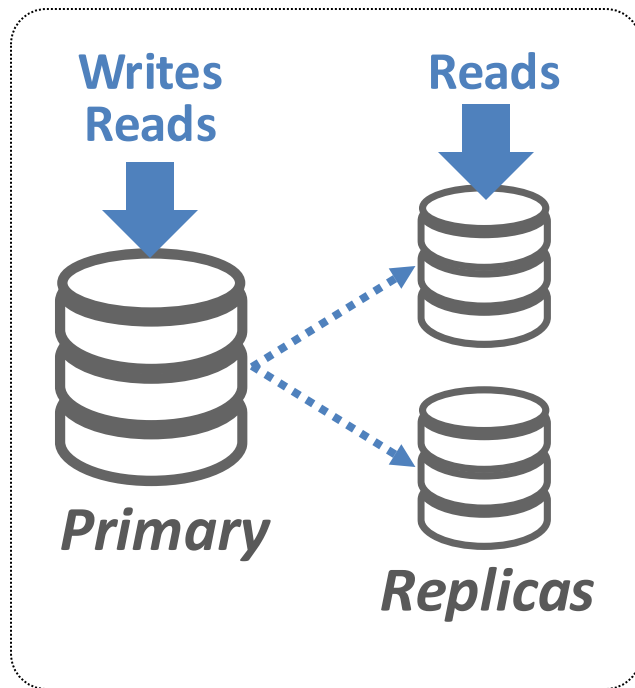
- The DBMS can replicate a database across redundant nodes to increase availability.
- Design Decisions:
 - Replica Configuration
 - Propagation Scheme
 - Propagation Timing
 - Update Method

Replica Configuration

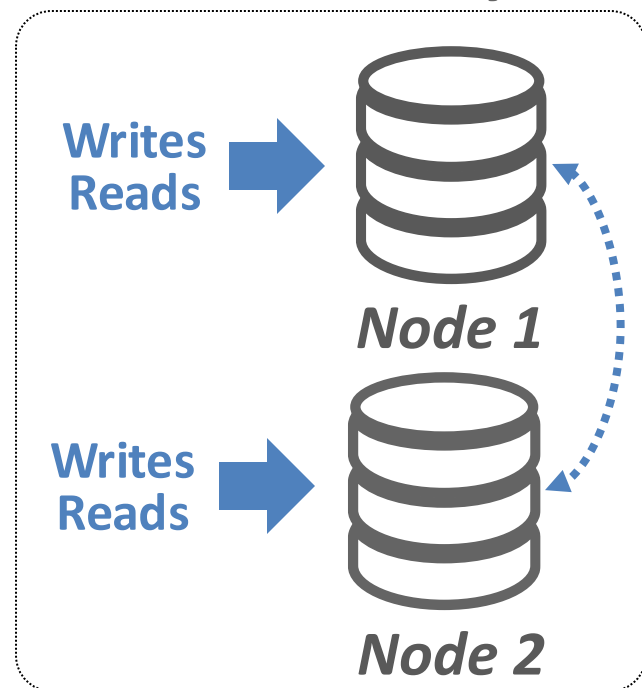
- **Approach #1: Primary-Replica**
 - All updates go to a designated primary for each object.
 - The primary propagates updates to its replicas by shipping logs.
 - Read-only txns may be allowed to access replicas.
 - If the primary goes down, recover by selecting a new primary.
- **Approach #2: Multi-Primary**
 - Txns can update data objects at any replica.
 - Replicas must synchronize with each other.

Replica Configuration

Primary-Replica



Multi-Primary



K-Safety

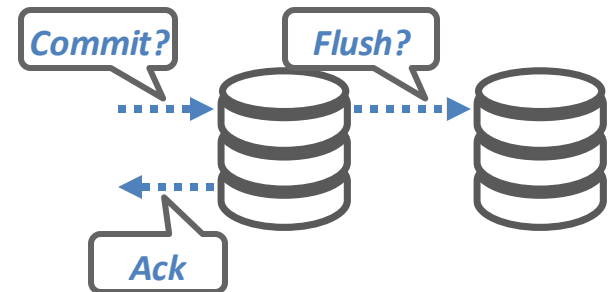
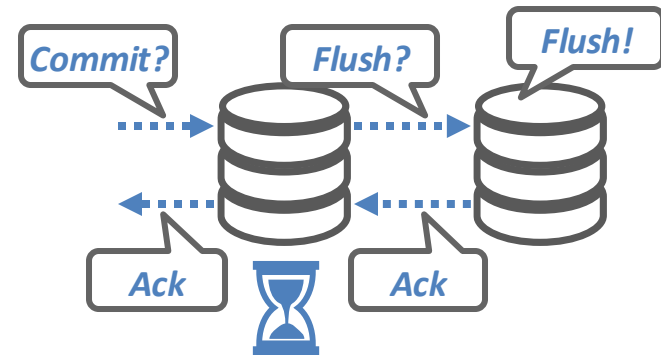
- ***K***-safety is a threshold for determining the fault tolerance of the replicated database.
- The value ***K*** represents the number of replicas per data object that must always be available.
- If the number of replicas goes below this threshold, then the DBMS halts execution and takes itself offline.

Propagation Scheme

- When a txn commits on a replicated database, the DBMS decides whether it must wait for that txn's changes to propagate to other nodes before it can send the acknowledgement to application.
- Propagation levels:
 - Synchronous (*Strong Consistency*)
 - Asynchronous (*Eventual Consistency*)

Propagation Scheme

- **Approach #1: Synchronous**
 - The primary sends updates to replicas and then waits for them to acknowledge that they fully applied (i.e., logged) the changes.
- **Approach #2: Asynchronous**
 - The primary immediately returns the acknowledgement to the client without waiting for replicas to apply the changes.



Propagation Timing

- **Approach #1: Continuous**
 - The DBMS sends log messages immediately as it generates them.
 - Also need to send a commit/abort message.
- **Approach #2: On Commit**
 - The DBMS only sends the log messages for a txn to the replicas once the txn is commits.
 - Do not waste time sending log records for aborted txns.

Update Method

- **Approach #1: Active-Active**
 - A txn executes at each replica independently.
 - Must be careful that transaction execution is deterministic or convergent.
- **Approach #2: Active-Passive**
 - Each txn executes at a single location and propagates the changes to the replica.
 - Can either do physical or logical replication.
 - Not the same as Primary-Replica vs. Multi-Primary

Distributed Transactions

- With partitioning / multi-primary replication, transactions must be ACID across distributed boundaries
- This makes transactions much harder because we must now handle distributed system anomalies
- Next class: Distributed Transactions